

Stateless vs Stateful ウィジェット

StatelessWidget と StatefulWidget を理解する

a. StatelessWidget と StatefulWidget とは？

- Flutter には主に 2 種類のウィジェットがある: StatelessWidget と StatefulWidget.
- どちらも UI を構築するために使用されるが、動作が異なる.
- Stateless widget は構築後に変更されず, Stateful widget は動的に変化する.

b. StatelessWidget とは？

- StatelessWidget は、一度作成されたら変更されない.
- Flutter が明示的に再構築しない限り、状態はそのまま.
- 静的なコンテンツ (テキスト、アイコン、画像など) の表示に最適.

```
class TitleWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text('Welcome to My App');  
  }  
}
```

- 説明:

- ◇ ウィジェットは常に「Welcome to My App」と表示される.
- ◇ アプリ内で何が起こっても、このテキストは変わらない.

c. StatefulWidget とは？

- StatefulWidget は時間とともに変化する.
- UI 要素が動的に更新される必要がある場合に使用.
- よく使われる場面: ボタン、フォーム、ユーザーインタラクション.

```
class ButtonWidget extends StatefulWidget {  
  @override  
  _ButtonWidgetState createState() =>  
    _ButtonWidgetState();  
}  
  
class _ButtonWidgetState extends State<ButtonWidget> {  
  String buttonText = 'Press me';  
  
  void updateText() {  
    setState(() {  
      buttonText = 'You pressed the button!';  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {
```

```

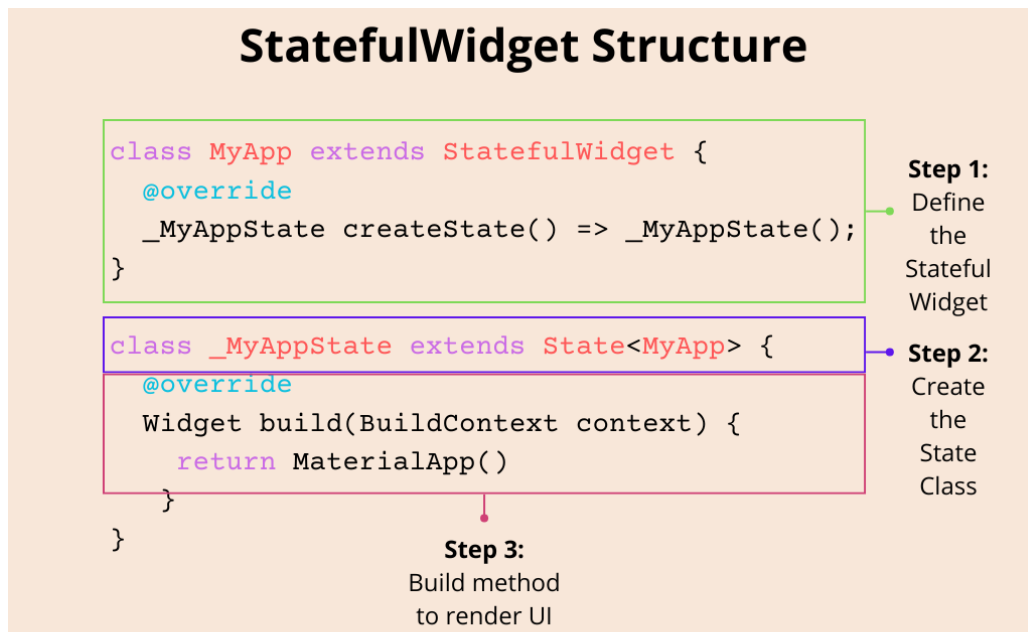
        return ElevatedButton(
            onPressed: updateText,
            child: Text(buttonText),
        );
    }
}

```

■ 説明:

- ◇ `_ButtonWidgetState` は, `ButtonWidget` の状態を 管理するクラス です.
- ◇ `State<ButtonWidget>` を継承しており, `ButtonWidget` というウィジェットに対応している.
- ◇ `build()` メソッドは オーバーライドされて, ウィジェットの見た目を定義する.
- ◇ `setState()` を呼び出すことで, UI が更新される.

d. `StatefulWidget` の構造



■ 構造の説明:

ステップ	起こること	目的	例
1	特別なウィジェット型を継承するクラスを定義する	状態を持つ動作を可能にする	<pre>class ButtonWidget extends StatefulWidget</pre>
2	ロジックコントローラーと接続するメソッドを提供する	状態を保持するオブジェクトを返す	<pre>@override _ButtonWidgetState createState() => _ButtonWidgetState();</pre>

3	変更を管理するための2つ目のクラスを作成する	ウィジェットがどのように反応するかを決定する	<pre>class _ButtonWidgetState extends State<ButtonWidget> {}</pre>
4	UIの更新を行う特別なメソッドを使う	Flutterにウィジェットの再構築を通知する	<pre>setState(() { buttonText = 'Updated!'; });</pre>

- ◇ ウィジェットクラスの中には、スーパークラスから再定義されたメソッドが存在する。
- ◇ そのメソッドの戻り値は、ロジックを処理する別のクラスのインスタンス。
- ◇ ロジック処理クラスは常に、ウィジェットを型引数として受け取る別のクラスを継承している。
- ◇ ビルド構造は、そのロジック処理クラスの中で定義され、ウィジェットを構築して返すメソッドによって記述される。

e. StatelessWidget と StatefulWidget の違い

■ 比較表

特徴	StatelessWidget	StatefulWidget
状態が変わるか?	いいえ	はい
状態管理	内部状態なし	内部状態あり
パフォーマンス	より効率的	効率が悪い
例	静的テキスト, アイコン, 画像	インタラクティブなボタン, フォーム
ウィジェットクラス	<pre>class MyWidget extends StatelessWidget</pre>	<pre>class MyWidget extends StatefulWidget</pre>
構築方法	build() のみ必要	createState() + State クラスが必要

f. 比較: Java (Android 開発) との違い

- Java (Android) では、UI コンポーネントの更新に View と Listener を使用する。

```
TextView textView = findViewById(R.id.myTextView);
textView.setText("Welcome to My App");
```

```
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        button.setText("You pressed the button!");
    }
});
```

- Flutter では、宣言的に setState() を使って UI を更新する。